Atividade Mineração de dados

Bacharelado em Estatística · UFMT

Geovani Carvalho Pereira

Sumário

1 **Questão 1.1** 1
2 **Questão 1.2** 2
3 **Questão 1.3** 3
4 **Questão 1.4** 3
5 **Questão 1.5** 5
6 **Questão 1.6** 5
6.1 **Diferença entre Concatenar e Juntar bases de dados** 7
7 **Questão 1.7** 7
8 **Questão 1.8** 8
9 **Questão 1.9** 9

1 Questão 1.1

a) O que é mineração de dados?

A **mineração de dados** (*data mining*), popularmente tratada como sinônimo de **descoberta de conhecimento a partir de dados** (*Knowledge Discovery from Data* – KDD), é o processo de extração automatizada de padrões e conhecimentos implícitos, previamente desconhecidos, válidos e potencialmente úteis a partir de grandes volumes de dados armazenados em repositórios digitais

b) **Qual a relação entre mineração de dados, Estatística, Aprendizado de Máquina e Bancos de Dados?**

A mineração de dados é uma área essencialmente **interdisciplinar** que integra conceitos dessas três frentes:

- **Bancos de Dados:** Fornecem a infraestrutura de suporte para armazenamento, indexação, processamento de consultas eficientes e gerenciamento escalável de grandes bases de dados e *data warehouses*.
- **Aprendizado de Máquina (Machine Learning):** Fornece os algoritmos para aprendizado automatizado (supervisionado e não supervisionado), permitindo a indução de modelos como árvores de decisão, redes neurais e agrupamentos.
- **Estatística:** Fornece a fundamentação matemática para sumarização de dados, modelagem probabilística, análise de correlação, regressão e validação da significância estatística dos padrõe

c) Descreva as principais etapas do processo de descoberta de conhecimento em bases de dados, indicando o objetivo de cada uma delas.

O KDD é um processo iterativo composto por 7 etapas encadeadas:

1. **Limpeza de Dados (Data Cleaning):** Elimina ruídos, inconsistências e erros da base.
2. **Integração de Dados (Data Integration):** Combina dados provenientes de múltiplas fontes heterogêneas.

3. **Seleção de Dados (Data Selection):** Recupera os subconjuntos de dados relevantes para o objetivo da análise.
4. **Transformação de Dados (Data Transformation):** Consolida e adequa os dados para o formato exigido pelas técnicas analíticas (por meio de normalização, agregação ou discretização).
5. **Mineração de Dados (Data Mining):** Aplicam-se métodos inteligentes para extrair padrões e modelos dos dados organizados.
6. **Avaliação de Padrões (Pattern Evaluation):** Filtra os padrões descobertos para identificar apenas os que são verdadeiramente interessantes e úteis, utilizando medidas objetivas e subjetivas de interesse.
7. **Apresentação do Conhecimento (Knowledge Presentation):** Utiliza técnicas de visualização gráfica para comunicar os resultados ao especialista ou tomador de decisão

d) Explique por que a mineração de dados não deve ser entendida apenas como a aplicação de um algoritmo a um conjunto de dados.

A mineração de dados em si corresponde a apenas **uma das etapas** (a quinta etapa) do processo KDD. Aplicar um algoritmo diretamente sobre dados brutos tende a falhar, pois dados do mundo real costumam ser incompletos, com ruídos e inconsistentes, o que levaria a resultados incorretos (*"garbage in, garbage out"*). Sem o **pré-processamento** prévio (limpeza, seleção e transformação) e sem a **avaliação e apresentação posterior** dos resultados, os algoritmos geram apenas equações ou agrupamentos mecânicos, que não se traduzem em conhecimento acionável para o negócio ou pesquisa

2 Questão 1.2

a) Diferencie dados estruturados, semiestruturados e não estruturados.

- **Dados Estruturados:** Dados organizados sob um esquema fixo, rígido e pré-definido, organizados em tabelas com linhas (registros) e colunas (atributos). Cada atributo possui um tipo de dado bem delimitado (ex: número, data, texto).
- **Dados Semiestruturados:** Dados que não possuem a rigidez de uma tabela relacional, mas contêm **marcadores, tags ou rótulos internos** (auto-descritivos) que organizam os elementos e hierarquias. Permitem campos flexíveis ou aninhados, onde nem todos os registros precisam ter os mesmos atributos.
- **Dados Não Estruturados:** Dados em formato bruto que **não possuem uma estrutura organizacional ou esquema pré-definido**. O conteúdo é livre ou conceitual, exigindo técnicas de interpretação/extração para identificar seu significado.

b) Apresente um exemplo de fonte de dados para cada um desses tipos.

- **Estruturados:** Tabelas de bancos de dados relacionais (SQL), arquivos CSV ou *Data Warehouses* contendo cadastros de clientes e histórico de vendas.
- **Semiestruturados:** Arquivos nos formatos **JSON**, **XML**, **HTML** ou documentos em bancos de dados NoSQL orientados a documentos (ex: logs de navegação web).
- **Não Estruturados:** Documentos de texto livre (PDFs, e-mails, postagens em redes sociais), imagens, arquivos de áudio, vídeos de vigilância e sinais de sensores.

c) Explique como o tipo e a estrutura dos dados podem influenciar as etapas de preparação e análise.

- **Na etapa de Preparação (Pré-processamento):** Dados estruturados passam por processos diretos de limpeza (tratamento de nulos/outliers) e normalização, pois as consultas SQL e tabelas facilitam a manipulação.Dados semiestruturados exigem etapas de parsing (decodificação de tags XML/JSON) e conversão (flattening) para transformar estruturas hierárquicas e aninhadas em matrizes/tabelas analíticas.Dados não estruturados demandam o uso de engenharia/extração de características

(feature extraction). Por exemplo, textos precisam de técnicas de Processamento de Linguagem Natural (NLP) como tokenização, remoção de stop-words e geração de embeddings; imagens exigem transformação gráfica e extração de bordas/texturas.

- **Na etapa de Análise (Algoritmos e Custo):** Estruturados podem ser analisados diretamente por algoritmos tradicionais de mineração (árvores de decisão, k-means, regressões).Não estruturados e semiestruturados exigem arquiteturas de aprendizado mais complexas e computacionalmente intensivas (como Redes Neurais Profundas e Aprendizado Profundo), além de infraestruturas de armazenamento especializadas (como Data Lakes e ecossistemas Big Data).

3 Questão 1.3

a) Pré-processamento de Dados

- **Conceito:** É o termo guarda-chuva amplo que engloba todas as atividades voltadas para a preparação e melhoria da qualidade dos dados brutos antes do processo de mineração. Como os dados do mundo real tendem a ser incompletos, com ruídos e inconsistentes, o pré-processamento busca tornar a análise mais acurada e eficiente.
- **Exemplos de procedimentos:**
  1. **Tratamento de valores ausentes (missing values):** Preenchimento de dados faltantes utilizando estimativas (como a média/mediana da classe, regressão ou inferência por árvores de decisão) ou remoção controlada de registros.
  2. **Suavização de ruídos (data smoothing):** Aplicação de técnicas de agrupamento por caixas (*binning*), regressão ou clusterização para filtrar variações aleatórias e erros pontuais de medição

b) Transformação de DadosConceito: É a etapa em que os dados são convertidos, reescalonados ou consolidados em formatos apropriados para que os algoritmos analíticos funcionem adequadamente ou interpretem os padrões de forma mais simples. Exemplos de procedimentos: **Normalização:** Reescalonamento das variáveis numéricas para intervalos comuns (como ou $[-1, 1]$ por min-max ou z-score), prevenindo que atributos com amplitudes maiores dominem os cálculos de distância (fundamental para algoritmos como k-NN, redes neurais e clusterização). **Discretização / Geração de Hierarquias de Conceitos:** Conversão de atributos numéricos contínuos em intervalos ou rótulos qualitativos (por exemplo, transformar valores contínuos de idade em faixas como "jovem", "adulto" e "idoso"

c) Redução de Dados

- **Conceito:** Conjunto de estratégias que obtém uma representação compacta do conjunto de dados, significativamente menor em volume (reduzindo o número de linhas ou de colunas), mas que preserva a integridade da informação e produz resultados analíticos equivalentes.
- **Exemplos de procedimentos:**
  1. **Redução de Dimensionalidade (Seleção de Atributos ou PCA):** Eliminação de variáveis irrelevantes/redundantes (*Attribute Subset Selection*) ou combinação de atributos em componentes sintéticos de menor dimensão (*Análise de Componentes Principais - PCA*).
  2. **Redução de Numerosidade (Amostragem ou Agregação):** Substituição do volume total de dados por modelos representativos ou amostras (como amostragem estratificada, histogramas ou agregação em cubos de dados OLAP)

---

4 Questão 1.4

---

a) Registros duplicados (*Tuple Duplication*) * **Impacto na análise:** A presença de linhas idênticas ou redundantes distorce estatísticas descritivas (infla artificialmente frequências e médias) e pode

superestimar a relevância de certos padrões em regras de associação e modelos de aprendizado, além de desperdiçar recursos computacionais. * **Estratégias de tratamento:** 1. **Identificação por chaves únicas:** Checagem de duplicatas exatas através do identificador primário (*ID*) do registro. 2. **Verificação de similaridade (*Deduplication/Entity Resolution*):** Uso de algoritmos de comparação de campos (*fuzzy matching*) quando há pequenas variações de digitação entre registros que representam a mesma entidade. 3. **Remoção de duplicatas:** Manutenção de apenas uma ocorrência válida do registro no conjunto de dados.

---

b) Valores ausentes (*Missing Values*) * **Impacto na análise:** Inviabiliza a execução de algoritmos que não suportam campos nulos (como regressões, redes neurais e (k)-means), reduz o tamanho efetivo da amostra e pode introduzir viés analítico caso a ausência não seja aleatória. * **Estratégias de tratamento:** 1. **Descarte/Remoção (*Listwise Deletion*):** Eliminação da linha ou coluna, recomendada quando o percentual de faltantes é muito pequeno ou quando o registro possui a maioria dos atributos nulos. 2. **Imputação por Tendência Central:** Preenchimento dos nulos utilizando a **média** (para distribuições simétricas) ou a **mediana** (para dados assimétricos) nos atributos numéricos, e a **moda** em atributos categóricos. 3. **Imputação Baseada em Modelos:** Estimação do valor faltante utilizando algoritmos preditivos (como (k)-NN, árvores de decisão ou regressão) com base nos outros atributos. 4. **Criação de Categoria "Desconhecido" ou Indicador:** Atribuição de um rótulo explícito (ex: `"Não Informado"`) para preservar a informação da ausência.

---

c) Valores atípicos (*Outliers*) * **Impacto na análise:** Distorcem fortemente métricas paramétricas (como média e variância) e afetam negativamente algoritmos baseados em distância (como (k)-means e (k)-NN) ou modelos lineares, fazendo com que o algoritmo tente se ajustar a ruídos pontuais em vez da tendência geral. * **Estratégias de tratamento:** 1. **Detecção Estatística/Visual:** Identificação via **Boxplots** e intervalo interquartil (método IQR: valores fora de (Q_1 - 1.5 IQR) até (Q_3 + 1.5 IQR)), **Z-score** (valores com ($|z| > 3$)) ou técnicas de clusterização/densidade (como LOF). 2. **Remoção:** Exclusão do registro caso o *outlier* seja fruto de erro de medição ou digitação comprovado. 3. **Winsorização / Truncamento:** Substituição dos valores extremos pelos limites superior ou inferior aceitáveis. 4. **Transformação de Variáveis:** Aplicação de transformações como (como $\log(x)$) para reduzir a assimetria da distribuição. 5. **Análise Separada:** Em aplicações como detecção de fraudes, o *outlier* é o foco do estudo e deve ser isolado para análise direcionada.

---

d) Inconsistências na representação de categorias * **Impacto na análise:** O mesmo conceito é fragmentado em múltiplas categorias falsas por problemas de digitação ou padronização (ex: `"Mato Grosso"`, `"MT"`, `"mato grosso"`, `"Mato Grosso "`), gerando ruído, dispersão das frequências e perda de poder preditivo do modelo. * **Estratégias de tratamento:** 1. **Padronização de Texto:** Conversão de todas as *strings* para caixa baixa (ou alta), remoção de acentos e eliminação de espaços em branco nas extremidades (*trimming*). 2. **Mapeamento de Categoria (Dicionário "De-Para"):** Substituição sistemática de sinônimos, abreviações e variações por uma convenção única padronizada.

---

e) Inconsistências de formato ou tipo das variáveis * **Impacto na análise:** Armazenar dados numéricos ou datas como texto (*string*) impede a realização de cálculos matemáticos, operações de ordenação temporal ou o correto parsing por bibliotecas de mineração. Formatos heterogêneos de datas (ex: `"2025/12/25"` misturado com `"25/12/2025"`) geram falhas de processamento. * **Estratégias de tratamento:** 1. **Conversão Explicita de Tipos (*Casting*):** Conversão forçada das colunas para os tipos primitivos corretos (inteiro, flutuante, booleano, data/hora). 2. **Padronização de Datas:** Uniformização de todos os registros temporais sob um padrão único (como a norma ISO 8601: `YYYY-MM-DD`).

3. **Higienização de Caracteres Especiais:** Remoção de símbolos monetários (ex: `"R$"`), separadores de milhar incorretos ou unidades de medida antes de converter para número.

---

5 Questão 1.5

a) O que caracteriza um conjunto de dados no formato *tidy*? O conceito de **Tidy Data** (dados organizados/limpos), formalizado por Hadley Wickham, caracteriza um padrão conceitual e prático de estruturação de dados em que a **estrutura física da tabela reflete diretamente a sua estrutura semântica**. Em um conjunto de dados *tidy*, a organização tabular segue regras estritas e padronizadas que eliminam ambiguidades comuns encontradas em planilhas e tabelas desorganizadas (como nomes de colunas contendo valores em vez de variáveis ou múltiplos valores em uma única célula).

---

b) Qual deve ser a relação entre variáveis, observações e valores na organização da tabela? Na estrutura *tidy*, a relação trina entre variáveis, observações e valores é definida por três regras fundamentais: 1. **Cada variável forma uma coluna:** Todas as medições do mesmo atributo (ex: `idade`, `temperatura`, `data`) ficam reunidas exclusivamente em uma única coluna. 2. **Cada observação forma uma linha:** Todas os valores medidos sobre uma mesma unidade analítica ou indivíduo em um determinado instante/evento ficam reunidos em uma única linha. 3. **Cada valor ocupa uma célula:** A intersecção de uma linha e uma coluna contém apenas um único dado/valor primitivo.

(Adicionalmente, se houver diferentes tipos de unidades observacionais no projeto, cada tipo deve ser armazenado em sua própria tabela distinta).

---

c) Por que essa estrutura facilita as etapas posteriores de manipulação, visualização e análise dos dados? Adotar o padrão *tidy* traz diversas vantagens para o fluxo analítico: * **Padronização e Consistência:** Permite desenvolver uma linguagem e ferramentas de manipulação uniformes (como as bibliotecas `pandas` em Python ou `dplyr` em R), já que todas as funções esperam receber as variáveis dispostas em colunas. * **Vetorização de Operações:** Linguagens de programação analíticas operam eficientemente sobre vetores. Como cada coluna representa uma variável inteira, é fácil aplicar transformações matemáticas, filtragens e cálculos diretamente sobre as colunas sem necessidade de laços (*loops*) manuais. * **Integração com a Gramática dos Gráficos:** Ferramentas de visualização (como `ggplot2` ou `seaborn`) mapeiam elementos visuais (eixo X, eixo Y, cor, tamanho, facetas) diretamente para as colunas da tabela. Dados no formato *tidy* podem ser passados diretamente para as funções de plotagem sem necessidade de reestruturações manuais a cada novo gráfico. * **Redução de Erros e Facilidade de Junção (*Joins*):** Evita inconsistências causadas por formatos "largos" (*wide format*) — onde datas ou anos viram títulos de colunas — facilitando o cruzamento entre diferentes bases de dados por meio de chaves relacionais simples.

---

6 Questão 1.6

Na construção de uma base analítica de dados, as operações de manipulação permitem reestruturar, limpar e agregar os dados para prepará-los para etapas subsequentes de análise e modelagem.

a) Selecionar variáveis (*Select*) * **Como funciona:** Seleciona ou descarta colunas específicas da tabela com base em seus nomes, posições ou tipos de dados. * **Para que serve:** Reduz a largura do conjunto de dados, eliminando variáveis irrelevantes ou redundantes para a análise, o que economiza memória e otimiza o tempo de processamento.

---

b) Filtrar observações (*Filter*) * **Como funciona:** Seleciona um subconjunto de linhas (observações) da tabela que satisfazem uma ou mais condições lógicas baseadas nos valores de suas variáveis (ex: `Estado == "MT"` ou `Idade >= 18`). * **Para que serve:** Delimita o escopo do estudo para a população de interesse e remove registros inconsistentes ou fora do domínio analítico.

---

c) Ordenar observações (*Arrange* / *Sort*) * **Como funciona:** Reorganiza a sequência lógica das linhas com base nos valores de uma ou mais colunas, em ordem crescente ou decrescente. * **Para que serve:** Facilita a inspeção visual dos extremos (maiores e menores valores), a identificação de *outliers* e a organização de séries temporais ou sequenciais.

---

d) Criar ou transformar variáveis (*Mutate* / *Derive*) * **Como funciona:** Computa novas colunas ou modifica colunas existentes aplicando transformações matemáticas, funções lógicas, conversões de formato ou discretizações sobre os dados originais. * **Para que serve:** Realiza a **engenharia de recursos** (*feature engineering*), permitindo calcular taxas, razões, diferenças de tempo ou escalas normalizadas necessárias para a modelagem.

---

e) Agrupar e resumir dados (*Group By & Summarize* / *Aggregate*) * **Como funciona:** Divide a tabela em subgrupos com base nos valores discretos de uma ou mais variáveis categóricas (*Group By*) e, em seguida, aplica funções de agregação (*Summarize* — como média, soma, contagem, mediana) sobre cada subgrupo. * **Para que serve:** Reduz a granularidade dos dados, permitindo obter estatísticas descritivas sumarizadas por categoria (ex: média de acidentes por município).

---

f) Identificar e remover registros duplicados (*Deduplication* / *Distinct*) * **Como funciona:** Examina a tabela em busca de linhas idênticas (duplicatas exatas em todas as colunas) ou de registros que compartilham a mesma chave primária. * **Para que serve:** Garante a unicidade das observações, prevenindo distorções em contagens e médias calculadas sobre a base.

---

g) Concatenar bases de dados (*Stacking* / *Union* / *Bind Rows/Cols*) * **Como funciona:** Empilha tabelas **verticalmente** (adicionando mais linhas/observações) ou as junta **horizontalmente** (adicionando colunas de igual extensão), sem exigir a busca por correspondência de chaves lógicas. * **Para que serve:** Unificar dados fragmentados que possuem exatamente a mesma estrutura (ex: juntar relatórios mensais de vendas de um mesmo ano).

---

h) Juntar ou conciliar bases de dados (*Joins* / *Merging*) * **Como funciona:** Combina colunas de duas ou mais tabelas relacionais cruzando e alinhando os registros por meio de uma ou mais **chaves de ligação** (*keys*). Os tipos mais comuns incluem: * *Inner Join:* Retorna apenas os registros com chaves coincidentes em ambas as tabelas. * *Left/Right Join:* Mantém todas as linhas da tabela da esquerda (ou direita) e traz as correspondências da outra. * *Full Outer Join:* Mantém todas as linhas de ambas as tabelas. * **Para que serve:** Enriquecer a base analítica combinando informações de origens distintas que descrevem as mesmas entidades.

6.1 Diferença entre Concatenar e Juntar bases de dados

Aspecto	Concatenar (*Bind / Union*)	Juntar / Conciliar (*Join / Merge*)
Mecanismo de combinação	Colagem direta por **posição estrutural** (empilhamento de linhas ou colateral de colunas).	Associação por **chave relacional** (busca de valores correspondentes entre identificadores).
Exigência prévia	Requer que as tabelas tenham os **mesmos nomes/tipos de colunas** (se vertical) ou o **mesmo número exato de linhas** (se horizontal).	Requer apenas uma ou mais **chaves de ligação em comum** (`ID`, `CPF`, `Código ICAO`, etc.).
Objetivo principal	Unir partes de uma mesma estrutura fragmentada (ex: agregar arquivos de Jan, Fev, Mar).	Cruzar atributos provenientes de tabelas totalmente diferentes sobre uma mesma entidade.

---

7 Questão 1.7

a) Em quais etapas desse processo a visualização pode ser utilizada? A visualização de dados pode ser empregada em **praticamente todas as etapas** do processo de Descoberta de Conhecimento em Bancos de Dados (KDD): 1. **Compreensão dos Dados e Exploração Inicial:** Para inspecionar rapidamente distribuições e estruturas preliminares dos dados brutos. 2. **Pré-processamento e Preparação dos Dados (Limpeza, Transformação e Redução):** Para diagnosticar graficamente falhas de qualidade, como *outliers*, dados ausentes, distribuições altamente assimétricas e redundâncias entre atributos. 3. **Mineração de Dados / Modelagem:** Como suporte no acompanhamento do ajuste de modelos (ex: curvas de aprendizado, dendrogramas em clusterização e estruturas de árvores de decisão). 4. **Avaliação de Padrões:** Para verificar visualmente a validade, separabilidade de classes, matrizes de confusão, curvas ROC e qualidade dos grupos formados. 5. **Apresentação do Conhecimento:** Etapa final dedicada a sintetizar e comunicar os *insights* descobertos aos tomadores de decisão por meio de relatórios e painéis (*dashboards*).

---

b) Qual o papel da visualização durante a exploração e preparação dos dados? Durante as fases iniciais do KDD, a visualização atua como a ferramenta central da **Análise Exploratória de Dados (EDA)**: * **Identificação de Formas e Distribuições:** Gráficos como histogramas, diagramas de densidade e *boxplots* revelam a simetria, assimetria, bimodalidade e variabilidade das colunas. * **Detecção de Anomalias (*Outliers* e Ruídos):** Facilita a identificação imediata de pontos discrepantes fora dos limites esperados (ex: pontos isolados em *boxplots* ou dispersões). * **Mapeamento de Relacionamentos e Correlações:** Gráficos de dispersão (*scatter plots*) e matrizes de correlação mostram associações lineares ou não lineares entre variáveis. * **Orientação de Decisões de Pré-processamento:** Ajuda a definir se uma variável precisa de transformação matemática (como escala logarítmica para suavizar assimetrias) ou se há atributos redundantes que podem ser eliminados na redução de dimensionalidade.

---

c) Como a visualização pode auxiliar na interpretação e comunicação dos resultados obtidos? * **Tradução de Complexidade:** Converte estruturas e equações matemáticas abstratas geradas pelos algoritmos (como probabilidades, coeficientes de regressão e clusters de alta dimensão) em representações visuais intuitivas. * **Validação por Especialistas:** Permite que especialistas do domínio

analisem os padrões extraídos sob a ótica do conhecimento do negócio, julgando se as descobertas são válidas, inovadoras ou mero ruído. * **Redução da Carga Cognitiva:** Facilita a assimilação rápida das mensagens principais por gestores e públicos não técnicos, destacando tendências e comparações de forma direta. * **Suporte à Tomada de Decisão:** Apresenta evidências visuais claras e persuasivas que justificam ações estratégicas baseadas em dados.

---

8 Questão 1.8

A Gramática dos Gráficos (*Grammar of Graphics*) é um arcabouço teórico (formalizado por Leland Wilkinson e popularizado em R por Hadley Wickham no pacote `ggplot2`) que estabelece que **qualquer gráfico estatístico pode ser decomposto e construído em camadas independentes e combináveis**. Em vez de tratar os gráficos como figuras estáticas fechadas (como "gráfico de pizza" ou "gráfico de colunas"), a gramática permite mapear declarativamente os dados sobre propriedades visuais e espaciais.

Abaixo estão os papéis de cada um dos componentes fundamentais do modelo:

- **a) Dados (*Data*):** Representam a **base de dados primária** (organizada preferencialmente no formato *tidy*) de onde os valores e observações são extraídos para alimentar a visualização.
- **b) Mapeamentos estéticos (*Aesthetics* / *aes*):** Definem a **associação lógica entre as variáveis da tabela e os atributos visuais** do gráfico, como os eixos de posição ((x, y)), cores (`colour` e `fill`), tamanho (`size`), forma dos pontos (`shape`) ou transparência (`alpha`).
- **c) Objetos geométricos (*Geoms*):** Definem as **formas geométricas concretas que representam visualmente as observações** na tela (ex: `geom_point` para pontos, `geom_bar` para barras, `geom_line` para linhas e `geom_boxplot` para diagramas de caixa).
- **d) Facetas (*Facets*):** Gerenciam a **divisão do gráfico em múltiplos painéis menores** (*small multiples*), fracionando a visualização com base nos níveis de uma ou mais variáveis categóricas (ex: `facet_wrap` e `facet_grid`).
- **e) Escalas (*Scales*):** Controlam o **mapeamento numérico e gráfico preciso entre os valores dos dados e o espaço visual**, regulando os limites dos eixos, intervalos das marcas (*ticks*), gradientes de cor, paletas e legendas.
- **f) Sistema de coordenadas (*Coordinate System*):** Gerencia a **organização do espaço bidimensional** onde os objetos são projetados, como o sistema cartesiano clássico (`coord_cartesian`), eixos invertidos (`coord_flip`) ou coordenadas polares (`coord_polar`).
- **g) Temas (*Themes*):** Controlam o **acabamento estético não de dados do gráfico**, como estilo e cor do fundo, fontes tipográficas, formatação dos rótulos, linhas de grade e posição das legendas.

Por que uma mesma base de dados pode resultar em diferentes representações gráficas? Uma mesma base de dados pode produzir gráficos completamente diferentes porque **a representação final é fruto da combinação específica e modular das escolhas feitas em cada camada**: 1. **Alteração na Geometria:** Trocar uma camada de pontos (`geom_point`) por barras (`geom_bar`) altera a perspectiva visual de uma dispersão individual para um resumo de frequências agregadas. 2. **Reorganização do Mapeamento Estético:** Mapear uma variável categórica para a cor (`colour`) em vez de criar painéis separados (`facet_wrap`) altera a forma como o leitor compara as categorias. 3. **Mudança no Sistema de Coordenadas:** Aplicar um sistema de coordenadas polares (`coord_polar`) sobre um gráfico de barras empilhadas transforma visualmente a estrutura em um gráfico de pizza.

---

9 Questão 1.9

a) Realize uma inspeção inicial do banco de dados, identificando o número de observações e variáveis, os nomes das variáveis e seus respectivos tipos.

```
prf <- read_delim("dados/PRFocorrencias.csv",
                  delim = ";",
                  locale = locale(decimal_mark = ',', encoding = 'Latin1') |>
                     clean_names()
)
# Verificar dimensões (número de linhas e colunas)
dim(prf)
```

```
[1] 64567    30
```

```
# Inspecionar estrutura, nomes de colunas e tipos de dados
glimpse(prf)
```

```
Rows: 64,567
Columns: 30
$ id                    <dbl> 331730, 331804, 331815, 331823, 331843, 331844,…
$ data_inversa          <chr> "01/01/2021", "01/01/2021", "01/01/2021", "01/0…
$ dia_semana            <chr> "sexta-feira", "sexta-feira", "sexta-feira", "s…
$ horario               <time> 05:30:00, 08:05:00, 10:10:00, 12:30:00, 14:40:…
$ uf                    <chr> "SP", "AM", "CE", "RJ", "RJ", "BA", "PR", "GO",…
$ br                    <dbl> 116, 174, 222, 493, 393, 101, 277, 20, 470, 381…
$ km                    <dbl> 453.0, 937.0, 128.5, 18.0, 252.0, 818.1, 51.3, …
$ municipio             <chr> "REGISTRO", "MANAUS", "ITAPAJE", "MAGE", "BARRA…
$ causa_acidente        <chr> "Reação tardia ou ineficiente do condutor", "Re…
$ tipo_acidente         <chr> "Tombamento", "Colisão traseira", "Tombamento",…
$ classificacao_acidente <chr> "Com Vítimas Feridas", NA, "Com Vítimas Feridas…
$ fase_dia              <chr> "Amanhecer", "Pleno dia", "Pleno dia", "Pleno d…
$ sentido_via           <chr> "Crescente", "Crescente", "Crescente", "Decresc…
$ condicao_metereologica <chr> "Nublado", "Céu Claro", "Céu Claro", "Nublado",…
$ tipo_pista            <chr> "Dupla", "Simples", "Simples", "Simples", "Simp…
$ tracado_via           <chr> "Curva", "Reta", "Curva", "Curva", "Reta", "Cur…
$ uso_solo              <chr> "Não", "Não", "Não", "Sim", "Não", "Não", "Não"…
$ pessoas               <dbl> 1, 7, 3, 3, 4, 2, 4, 2, 9, 4, 5, 3, 2, 2, 1, 2,…
$ mortos                <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 0, 0, 0,…
$ feridos_leves         <dbl> 1, 1, 2, 1, 1, 1, 1, 0, 5, 1, 0, 0, 0, 0, 1, 0,…
$ feridos_graves        <dbl> 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 2, 0, 0, 1, 0, 0,…
$ ilesos                <dbl> 0, 5, 0, 1, 1, 0, 3, 1, 0, 1, 2, 1, 1, 1, 0, 2,…
$ ignorados             <dbl> 0, 1, 11, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0…
$ feridos               <dbl> 1, 1, 2, 1, 2, 1, 1, 0, 6, 2, 2, 0, 0, 1, 1, 0,…
$ veiculos              <dbl> 1, 5, 12, 3, 3, 2, 3, 2, 4, 3, 3, 3, 2, 2, 1, 2…
$ latitude              <dbl> -24.561681, -2.508068, -3.707626, -22.668712, -…
$ longitude             <dbl> -47.86752, -60.03643, -39.62351, -43.01863, -43…
$ regional              <chr> "SPRF-SP", "SPRF-AM", "SPRF-CE", "SPRF-RJ", "SP…
```

```
$ delegacia              <chr> "DEL05-SP", "DEL01-AM", "DEL04-CE", "DEL04-RJ",…
$ uop                    <chr> "UOP01-DEL05-SP", "UOP01-DEL01-AM", "UOP03-DEL0…
```

```
# filtrar para Mato Grosso(MT)
prf_mt <- prf |>
    filter(uf == "MT")

prf_totais <- prf_mt |>
    summarise(
        total_acidentes = n(),
        total_pessoas = sum(pessoas, na.rm = TRUE),
        total_feridos_leves = sum(feridos_leves, na.rm = TRUE),
        total_feridos_graves = sum(feridos_graves, na.rm = TRUE),
        total_mortos = sum(mortos, na.rm = TRUE),
        total_veiculos = sum(veiculos, na.rm = TRUE)
        )

prf_totais |> print(width = Inf)
```

```
# A tibble: 1 × 6
  total_acidentes total_pessoas total_feridos_leves total_feridos_graves
            <int>         <dbl>               <dbl>                <dbl>
1            2140          5731                1821                  582
  total_mortos total_veiculos
         <dbl>          <dbl>
1          223           5135
```

```
# Função para gerar tabela de frequências absoluta e relativa

gera_tabela_freq <- function(dados, variavel) {
    dados |>
        count({{variavel}}, sort = TRUE) |>
        mutate(freq_relativa = round((n /sum(n)) *100, 2))
}

# Frequências por Dia da Semana
  freq_dia <- gera_tabela_freq(prf_mt, dia_semana)
  freq_dia
```

```
# A tibble: 7 × 3
  dia_semana        n freq_relativa
  <chr>         <int>         <dbl>
1 sábado          372          17.4
2 domingo         362          16.9
3 sexta-feira     298          13.9
4 quinta-feira    286          13.4
5 segunda-feira   282          13.2
```

```
6 terça-feira     276          12.9
7 quarta-feira    264          12.3
```

```
# Frequências por Fase do Dia
  freq_fase <- gera_tabela_freq(prf_mt, fase_dia)
  freq_fase
```

```
# A tibble: 4 × 3
  fase_dia        n freq_relativa
  <chr>       <int>         <dbl>
1 Pleno dia    1201         56.1
2 Plena Noite   688         32.2
3 Anoitecer     155          7.24
4 Amanhecer      96          4.49
```

```
# Frequências por Classificação do Acidente
  freq_classificacao <- gera_tabela_freq(prf_mt, classificacao_acidente)
  freq_classificacao
```

```
# A tibble: 3 × 3
  classificacao_acidente     n freq_relativa
  <chr>                  <int>         <dbl>
1 Com Vítimas Feridas     1539         71.9
2 Sem Vítimas              418         19.5
3 Com Vítimas Fatais       183          8.55
```

```
# top 10 causas mais frequentes (volume de acidentes)
top10_frequentes <- prf_mt |>
    count(causa_acidente, sort = TRUE) |>
    head(10)

top10_frequentes
```

```
# A tibble: 10 × 2
   causa_acidente                                                 n
   <chr>                                                      <int>
 1 Reação tardia ou ineficiente do condutor                     252
 2 Acessar a via sem observar a presença dos outros veículos    168
 3 Ausência de reação do condutor                               162
 4 Condutor deixou de manter distância do veículo da frente     157
 5 Velocidade Incompatível                                      146
 6 Ingestão de álcool pelo condutor                             138
 7 Ultrapassagem Indevida                                       117
 8 Demais falhas mecânicas ou elétricas                          94
 9 Manobra de mudança de faixa                                   85
10 Condutor Dormindo                                             82
```

```r
# top 10 causas  associadas ao maior número de morte
top10_mortes <- prf_mt |>
    group_by(causa_acidente) |>
    summarise(total_mortos = sum (mortos, na.rm = TRUE)) |>
    arrange(desc(total_mortos)) |>
    head(10)

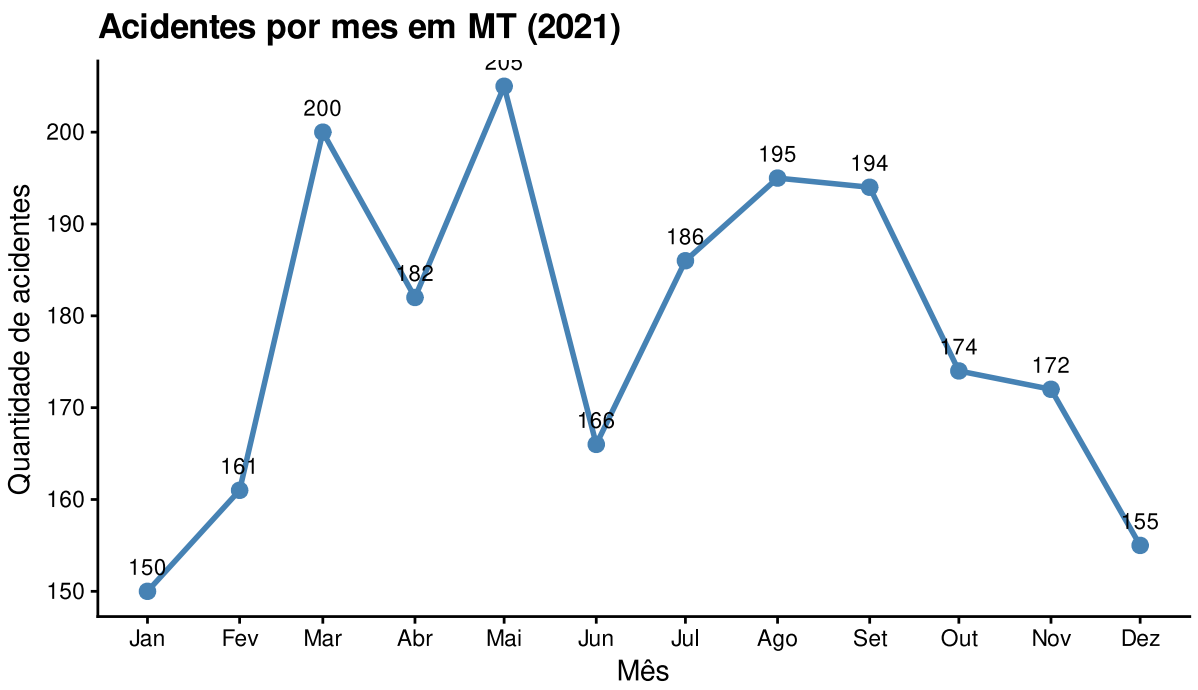
top10_mortes
```

```
# A tibble: 10 × 2
   causa_acidente                                              total_mortos
   <chr>                                                              <dbl>
 1 Transitar na contramão                                                41
 2 Ultrapassagem Indevida                                                28
 3 Ausência de reação do condutor                                        18
 4 Reação tardia ou ineficiente do condutor                              17
 5 Manobra de mudança de faixa                                           16
 6 Acessar a via sem observar a presença dos outros veículos             13
 7 Velocidade Incompatível                                               10
 8 Chuva                                                                  9
 9 Condutor Dormindo                                                      8
10 Condutor deixou de manter distância do veículo da frente               7
```

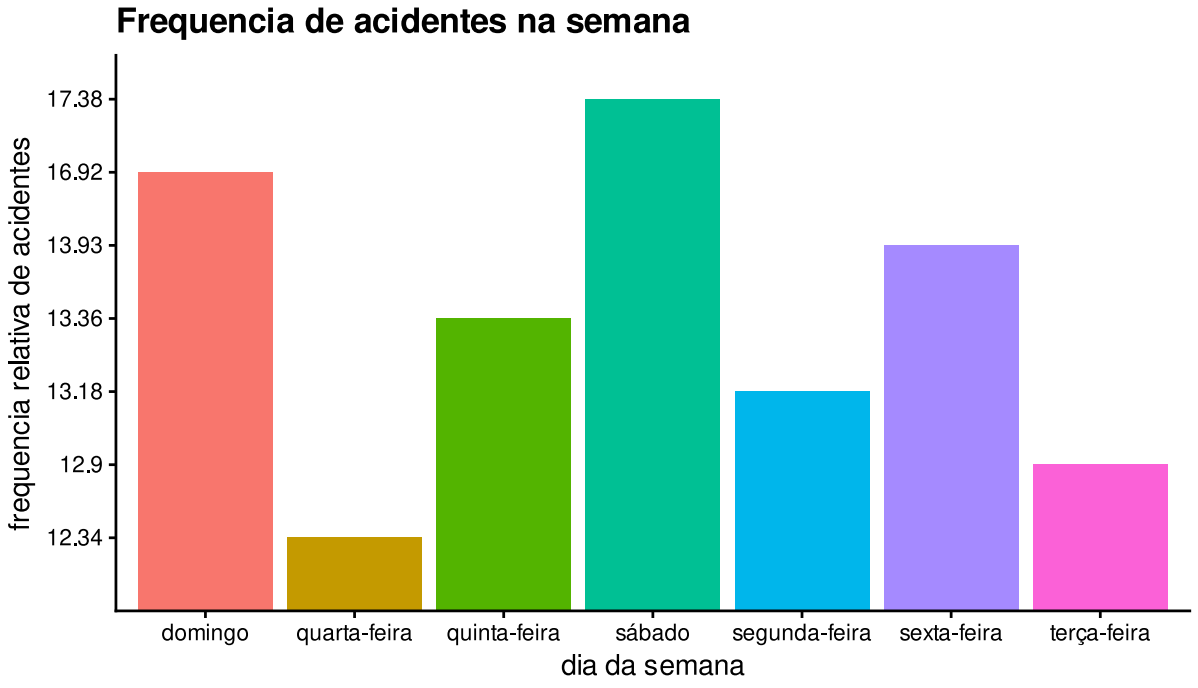
```r
# Serie temporal: acidentes por mes (jan-dez 2021)

# nomes de mes em portugues (independe do locale do sistema)
meses_pt <- c("Jan", "Fev", "Mar", "Abr", "Mai", "Jun",
              "Jul", "Ago", "Set", "Out", "Nov", "Dez")

prf_mt |>
  mutate(mes = floor_date(dmy(data_inversa), unit = "month")) |>
  count(mes) |>
  ggplot(aes(x = mes, y = n)) +
  geom_line(color = "steelblue", linewidth = 1) +
  geom_point(color = "steelblue", size = 2.5) +
  geom_text(aes(label = n), vjust = -0.8, size = 3) +
  scale_x_date(date_breaks = "1 month",
               labels = function(d) meses_pt[month(d)]) +
  labs(title = "Acidentes por mes em MT (2021)",
       x = "Mês", y = "Quantidade de acidentes")
```

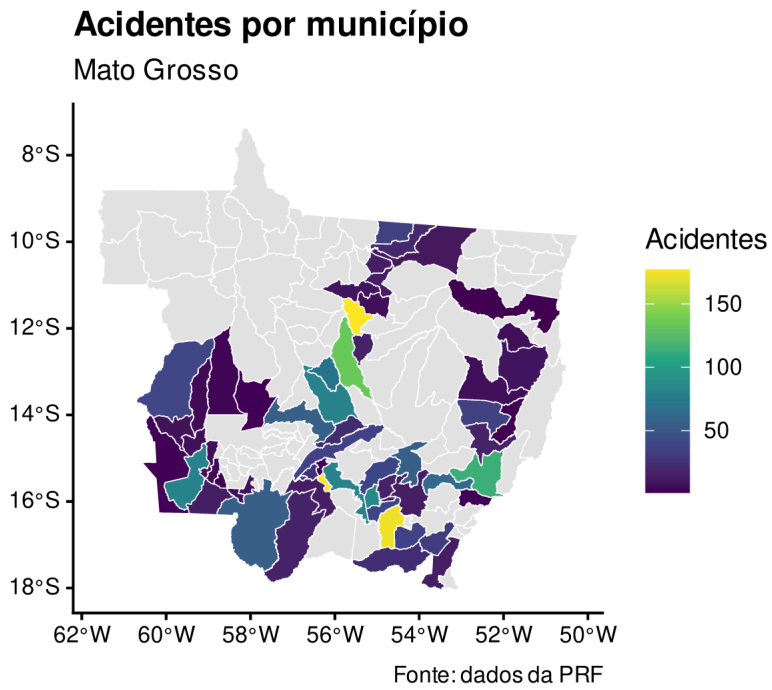



```
ggplot(freq_dia, aes(x = dia_semana, y = reorder(freq_relativa, n), fill =
dia_semana)) +
  geom_col() +
  labs(title = "Frequencia de acidentes na semana",
  x = "dia da semana", y = "frequencia relativa de acidentes") +
    theme(legend.position = "none")
```




```
ggplot(mapa_prf) +
  geom_sf(aes(fill = total_acidentes), color = "white") +
```

```
  scale_fill_viridis_c(
    name = "Acidentes",
    na.value = "grey90"
  ) +
  labs(
    title = "Acidentes por município",
    subtitle = "Mato Grosso",
    caption = "Fonte: dados da PRF"
  )
```



Em 2021, Mato Grosso registrou 2.140 acidentes nas rodovias federais, com 5.731 pessoas envolvidas, 5.135 veículos, 582 feridos graves e 223 mortos. A maior parte dos acidentes teve vítimas: 71,9% com feridos e 8,55% com vítimas fatais, enquanto apenas 19,5% não tiveram vítimas.

Os acidentes se concentram nos fins de semana: sábado (17,4%) e domingo (16,9%) lideram, enquanto os demais dias ficam entre 12,3% e 13,9%. Quanto ao período, 56,1% ocorreram em pleno dia e 32,2% à noite, sendo o amanhecer (4,49%) e o anoitecer (7,24%) os de menor frequência.

Entre as causas, as mais frequentes são falhas de atenção e reação do condutor: reação tardia ou ineficiente (252 casos), acesso à via sem observar outros veículos (168) e ausência de reação (162). Já as causas associadas a mais mortes são outras: transitar na contramão (41 mortes) e ultrapassagem indevida (28) aparecem no topo, apesar de menos frequentes, o que indica maior gravidade desses comportamentos. A ingestão de álcool (138 casos) e a velocidade incompatível (146) também estão entre as principais causas.

Espacialmente, os acidentes se concentram em poucos municípios, com destaque para alguns que passam de 150 ocorrências, enquanto grande parte do estado apresenta poucos registros.